

Rapporto Tecnico e di Sviluppo: Piattaforma di gestione Ticket

1. Introduzione e Obiettivi del Progetto

Il progetto ha avuto come obiettivo lo sviluppo di una **piattaforma web full-stack** per la gestione centralizzata delle richieste di supporto (ticket). La piattaforma è stata progettata per supportare due ruoli principali: **Utente Standard** (per l'apertura e la tracciabilità dei propri ticket) e **Amministratore** (per la supervisione, la gestione dei ticket e la manutenzione degli utenti).

Stack Tecnologico (Scelte Architettureali)

Componente	Tecnologia	Ruolo	Motivazione della Scelta
Backend	Node.js con Express	Gestione API REST	Flessibilità, prestazioni I/O non bloccanti e ampiezza dell'ecosistema JS.
Database	MySQL	Persistenza dei Dati	Affidabilità, relazioni strutturate e scalabilità per i dati relazionali (ticket, utenti).
Frontend	HTML, CSS, JavaScript (Vanilla)	Interfaccia Utente (UI)	Sviluppo rapido, leggerezza dell'applicazione e assenza di dipendenze da framework pesanti.

2. Struttura del Database e Sicurezza Fondamentale

Il database MySQL è stato progettato per garantire l'integrità referenziale e la gestione efficiente dei dati sensibili.

A. Modello Dati (Tabelle)

Sono state definite tre tabelle principali con relazioni chiare:

1. **users**: Memorizza i dati di autenticazione e i ruoli (`role: ENUM('user', 'admin')`).
2. **tickets**: Contiene il corpo della richiesta, lo stato (`aperto, in-corso, chiuso`) e la priorità. Relazione **uno-a-molti** con `users` (`user_id`).
3. **notes**: Documenta le comunicazioni interne (visibili solo agli Admin) relative a un ticket. Relazione **uno-a-molti** con `tickets` e `users` (autore della nota).

B. Sicurezza e Gestione dei Dati Personali

Per garantire la sicurezza dei dati la robustezza del sistema e la gestione controllata delle informazioni degli utenti sono state adottate diverse procedure

Misure di Sicurezza	Dettaglio Implementazione
Hashing Password	Utilizzo della libreria Bcrypt con un fattore di costo (<code>saltRounds = 10</code>) per criptare le password (<code>password_hash</code>). Impedisce a chiunque acceda al database (anche un amministratore) di conoscere la password originale dell'utente.
Prevenzione SQL Injection	Tutte le interazioni con il database avvengono tramite query parametrizzate , (<code>connection.query(sql, [values])</code>) in tutte le operazioni con il database, garantendo che l'input dell'utente sia sempre trattato come dati e mai come codice eseguibile.
Controllo d'Accesso	Il Login restituisce il <code>role</code> dell'utente, determinando l'accesso alla dashboard <code>admin</code> o <code>user</code> .

c. Conformità Normativa (GDPR)

Il progetto fornisce i **mezzi tecnici** per rispettare i requisiti fondamentali del GDPR, anche se la piena conformità è un tema legale e non solo tecnico.

Principio GDPR	Implementazione Tecnica nel Progetto	Stato di Conformità
Minimizzazione dei dati	Vengono raccolti solo name , email e password (ecco perché è importante non chiedere dati non necessari).	✓ Coperto
Integrità e Riservatezza	Uso di Bcrypt per le password e HTTPS (presupposto per un sito reale) per la trasmissione.	✓ Coperto
Diritto all'Oblio (Cancellazione)	Implementazione: La funzione DELETE /api/users/:userId elimina l'utente. Grazie alla clausola ON DELETE CASCADE nel database, i ticket e le note associate vengono eliminati automaticamente.	✓ Implementato
Trasparenza	Non è un requisito tecnico, ma richiede Informativa Privacy e Termini di Servizio chiare (da aggiungere nel footer di index.html e dashboard-user.html).	✗ Mancante (Legale/Documentale)

3. Funzionalità Core e Endpoint API

Le seguenti funzionalità cruciali sono state sviluppate tramite endpoint API REST in `server.js` e messe in atto dal frontend in `app.js`.

A. Autenticazione e Autorizzazione

Funzione	Route/Endpoint	Punti Salienti/Scelte di Design
Registrazione	POST /api/register	Implementata validazione e pulizia dell'input (<code>trim()</code>) per nomi, email e password prima dell'hashing.
Login	POST /api/login	Confronto con Bcrypt. Messaggio di errore generico (<code>Credenziali non valide</code>) per motivi di sicurezza contro l'enumerazione degli utenti .
Caricamento Profilo	GET /api/users/:userId	Aggiunta per risolvere un bug di visualizzazione; carica i dati utente attuali al caricamento della dashboard profilo.

B. Utente

Creazione Ticket	POST /api/tickets	Permette solo agli utenti <i>loggati</i> (con <code>user_id</code> valido) di inviare ticket.
------------------	-------------------	---

C. Gestione Admin (CRUD)

Funzione	Endpoint	Dettagli di Implementazione
Gestione Utenti	DELETE /api/users/:userId	Implementata l'eliminazione utente con un controllo di sicurezza per impedire l'eliminazione dell'admin primario.
Conformità GDPR	(DB Design)	Le relazioni DB sono definite con ON DELETE CASCADE , assicurando che l'eliminazione di un utente provochi l'eliminazione automatica dei suoi ticket e delle sue note (Diritto all'Oblio).
Note Interne	POST /api/notes	Permette all'admin di aggiungere commenti (non visibili all'utente) a un ticket specifico.

Questa è una **spiegazione dettagliata delle azioni principali che un utente standard compie** sul sito, analizzando il flusso interno che coinvolge Frontend (JavaScript in `app.js`), le chiamate API (Endpoint in `server.js` su Node.js) e l'interazione con il Database (MySQL).

Ci concentreremo esclusivamente sul flusso operativo dell'utente.

1. 🗝️ Registrazione di un Nuovo Utente

L'obiettivo è creare un nuovo record nella tabella `users` del database.

FASE	PROCESSO INTERNO	CAUSE ED EFFETTI
Frontend (<code>app.js</code>)	Azione: L'utente compila il form di registrazione in <code>index.html</code> e clicca su "Registrati" .	1. Raccolta Dati: <code>app.js</code> intercetta l'evento <code>submit</code> sul form (<code>id="register-form"</code>). I dati (<code>nome</code> , <code>email</code> , <code>password</code>) vengono raccolti.
Chiamata API	<code>fetch</code> (async): Viene eseguita una richiesta POST all'endpoint: <code>/api/register</code> . Il corpo della richiesta contiene i dati raccolti in formato JSON.	2. Trasmissione: I dati vengono inviati al backend (<code>server.js</code>).
Backend (<code>server.js</code>)	Route: <code>app.post('/api/register', ...)</code>	3. Validazione: Il server riceve i dati. Vengono eseguite operazioni di pulizia come <code>trim()</code> e controlli per garantire che tutti i campi obbligatori siano presenti.
Backend (Sicurezza)	Funzione Principale: <code>bcrypt.hash(password, saltRounds)</code>	4. Hashing: La password in chiaro viene criptata utilizzando la libreria Bcrypt (con <code>saltRounds = 10</code>). Viene generato l'hash sicuro (<code>password_hash</code>).
Database (MySQL)	Query: <code>INSERT INTO users (email, password_hash, name, role) VALUES (?, ?, ?, 'user')</code>	5. Inserimento: Viene eseguita una query parametrizzata per inserire il nuovo utente. Viene salvato l'hash della password (non la password originale) e il campo <code>role</code> viene impostato di default su 'user' .
Risposta e Finale	Il backend restituisce uno status 201 Created .	6. Reindirizzamento: <code>app.js</code> riceve la risposta positiva e reindirizza l'utente alla schermata di Login .

2. Accesso (Login)

L'obiettivo è autenticare l'utente confrontando la password fornita con l'hash salvato nel database.

FASE	PROCESSO INTERNO	CAUSE ED EFFETTI
Frontend (app.js)	Azione: L'utente compila il form di login in <code>index.html</code> e clicca su "Accedi".	1. Raccolta Dati: <code>app.js</code> raccoglie <code>email</code> e <code>password</code> .
Chiamata API	<code>fetch</code> (async): Viene eseguita una richiesta POST all'endpoint: <code>/api/login</code> .	2. Trasmissione: Le credenziali (in chiaro) vengono inviate al server.
Backend (server.js)	Route: <code>app.post('/api/login', ...)</code>	3. Ricerca Utente: Il server esegue la query <code>SELECT id, password_hash, role FROM users WHERE email = ?</code> per trovare l'utente tramite l'email.
Backend (Sicurezza)	Funzione Principale: <code>bcrypt.compare(password, user.password_hash)</code>	4. Confronto Password: Se l'utente esiste, la password fornita viene confrontata con la <code>password_hash</code> salvata usando Bcrypt .
Outcome (Successo)	La funzione <code>bcrypt.compare</code> restituisce <code>true</code> .	5. Autenticazione: Il server estrae l' <code>id</code> e il <code>role</code> dell'utente e li invia al frontend con status 200 OK .
Outcome (Fallimento)	<code>bcrypt.compare</code> restituisce <code>false</code> (o utente non trovato).	6. Errore Sicuro: Il server invia uno status 401 Unauthorized con un messaggio di errore generico (per non rivelare se l'email è sbagliata o solo la password).
Final Effect (Frontend)	Funzioni Principali: <code>localStorage.setItem('userId', id)</code> e <code>window.location.href = 'dashboard-user.html'</code>	7. Sessione: In caso di successo, <code>app.js</code> salva l' <code>id</code> e il <code>role</code> dell'utente nel browser (in <code>localStorage</code>) e reindirizza alla dashboard utente .

3. Creazione di un Nuovo Ticket

L'obiettivo è inserire un nuovo record nella tabella `tickets`, collegandolo all'utente corrente.

FASE	PROCESSO INTERNO	CAUSE ED EFFETTI
Frontend (<code>app.js</code>)	Azione: L'utente nella <code>dashboard-user.html</code> compila il form (<code>id="ticket-creation-form"</code>) e clicca "Invia ticket".	1. Raccolta Dati: <code>app.js</code> raccoglie <code>titolo</code> , <code>descrizione</code> e <code>priorità</code> .
Frontend (Dipendenza)	Funzione Principale: <code>localStorage.getItem('userId')</code>	2. Identificazione Utente: L' <code>id</code> dell'utente (salvato al login) viene recuperato da <code>localStorage</code> e incluso nei dati da inviare.
Chiamata API	<code>fetch</code> (async): Viene eseguita una richiesta POST all'endpoint: <code>/api/tickets</code> .	3. Trasmissione: I dati del ticket + l' <code>id</code> utente vengono inviati al server.
Backend (<code>server.js</code>)	Route: <code>app.post('/api/tickets', ...)</code>	4. Preparazione Dati: Il server riceve i dati. Viene verificata la presenza dello <code>user_id</code> e degli altri campi obbligatori.
Database (MySQL)	Query: <code>INSERT INTO tickets (user_id, title, description, priority, status) VALUES (?, ?, ?, ?, 'aperto')</code>	5. Inserimento: Viene eseguita una query parametrizzata . Il ticket viene inserito, e: a) <code>user_id</code> collega il ticket al creatore. b) Il campo <code>status</code> viene impostato di default su <code>'aperto'</code> .
Risposta e Finale	Il backend restituisce uno status 201 Created .	6. Refresh Vista: <code>app.js</code> resetta il form e invia una nuova richiesta (GET <code>/api/user-tickets/:userId</code>) per ricaricare e visualizzare il nuovo ticket nell'elenco "I tuoi ticket".

4. Controllo Qualità (QA) e Miglioramenti di Usabilità

Revisioni Più Importanti Durante Lo Sviluppo E Correzioni Mirate

Bug/Miglioramento	Descrizione del Problema	Soluzione (Modifiche CSS/JS)
Validazione Inconsistente	La logica di login non verificava la pulizia degli input come la registrazione, accettando anche password composte solo da spazi o input vuoti.	Estensione delle regole <code>trim()</code> su <code>email</code> , <code>name</code> , <code>password</code> e controllo sulla <code>length > 0</code> alla route <code>POST /api/login</code> .
Dati Profilo Errati	Dopo il login, il form profilo utente visualizzava i dati del precedente utente loggato. (problema di cache visiva).	Aggiunta la funzione <code>fetchUserProfile()</code> in <code>app.js</code> che recupera i dati utente tramite <code>GET /api/users/:userId</code> e popola i campi del form con i dati reali dell'utente loggato.
Overflow Testo	Testi lunghi (es. Titoli) fuoriuscivano dai confini delle celle, compromettendo il layout.	Applicazione delle proprietà <code>word-break: break-all;</code> (per Utente) e <code>overflow-wrap: break-word;</code> (per Titolo) per forzare il testo ad andare a capo e non fuoriuscire.

5. Sviluppi Futuri

Il progetto è ora in una fase stabile con un'infrastruttura solida e tutte le funzionalità CRUD di base per la gestione dei ticket e degli utenti.

Prossimi Sviluppi Consigliati:

- **Implementazione di JWT:** Adottare i JSON Web Tokens per una gestione dell'autenticazione più sicura e scalabile, limitando l'uso di `localStorage` per i dati utente non essenziali.
- Consentire di ordinare i ticket secondo utente o priorità, possibilità di eliminare le singole note interne.
- **Protezioni** contro Cross-Site Scripting (XSS) e/o altri attacchi
- **Responsività:** Ottimizzare il layout CSS per una migliore esperienza utente su dispositivi mobili.
- **Interfaccia Migliorata:** Utilizzare una libreria di componenti o un framework CSS (es. Tailwind o Bootstrap) per una gestione più efficiente degli stili e della responsività.

Osservazioni e Conclusioni

Se viene inserito codice SQL dannoso nel campo "Descrizione" e.g, la libreria mysql2 in Node.js lo tratterà semplicemente come testo e non lo eseguirà come comando. I valori [user_id, title, description, priority] sono passati come array separato. Questo attiva i Prepared Statements

L'altra forma comune di attacco che potrebbe avvenire tramite campi di testo (come la descrizione) è il Cross-Site Scripting (XSS). Questo accade quando un utente malintenzionato inserisce codice HTML o JavaScript nel campo, e questo codice viene poi visualizzato ed eseguito nel browser di un altro utente (ad esempio, un Admin che visualizza il ticket). Si potrebbe implementare la difesa contro l'XSS sul lato Frontend, durante la visualizzazione dei dati. In tutti i punti in cui si visualizzano contenuti generati dall'utente (come la descrizione del ticket o il contenuto della nota), usando la proprietà `.textContent` anziché `.innerHTML`.

Termini tecnici

Un **Endpoint** è l'URL specifico a cui il client (il tuo file `app.js` e i file HTML) invia una richiesta per accedere a una risorsa o eseguire un'azione sul server.

Un **Framework** è un insieme di strumenti, librerie, regole e convenzioni predefinite che fornisce una struttura di base su cui costruire applicazioni software (in questo caso [express.js](#)).

API REST (Architettura per la Comunicazione) **API** sta per Application Programming Interface. È un "contratto" che definisce come due pezzi di software devono interagire tra loro. **REST** sta per Representational State Transfer ed è uno *stile* architetturale per la progettazione di API. Un'API è definita "RESTful" se rispetta i principi REST.

Principi Fondamentali di un'API RESTful:

1. **Risorse:** Tutto nel sistema (utenti, ticket, note) è visto come una **risorsa** identificabile da un URL (es. `/api/tickets`).
2. **Metodi HTTP:** Utilizza i metodi HTTP standard per eseguire le operazioni CRUD (Create, Read, Update, Delete) sulle risorse: Metodo **POST** Invia nuovi dati (es. creare un nuovo ticket) | **GET** Recupera i dati (es. visualizzare un ticket o tutti gli utenti) | **PUT / PATCH** Modifica i dati esistenti (es. cambiare la priorità di un ticket) | **DELETE** Rimuove una risorsa (es. eliminare un utente)

3. **Stateless (Senza Stato):** Ogni richiesta dal client al server deve contenere tutte le informazioni necessarie per capire la richiesta stessa. Il server non salva informazioni sulla sessione del client tra una richiesta e l'altra.
4. **Client-Server:** La responsabilità del client (UI) e del server (dati e logica) sono separate. Questo permette di sviluppare e scalare i due lati in modo indipendente.

Una classe **Card** è un contenitore flessibile e auto-contenuto, spesso con bordi, sfondo e ombra per sembrare un foglio di carta o un riquadro sollevato.

JSON è un formato leggero per lo scambio di dati. È il formato standard con cui le API RESTful inviano e ricevono i dati. I dati sono rappresentati come coppie **chiave-valore**.